

OpenClaw / Bybit Capability & Permission Switch Plan v1

0. 文档目的

这份文档用于把当前已经讨论过的 **GUI、控制平面、学习能力、产品族能力开关、权限边界、运营看板、自我感知、汇报能力** 统一收敛成一份可持续的正式规划。

它的目标不是只服务当前 demo 阶段，而是要作为未来一段时间内 OpenClaw / Bybit 图形化控制系统的**总纲领文件**，避免后续深化讨论时遗忘已确定的要求。

这份计划同时回答三类问题：

1. **GUI 现在到底要做什么**
2. **哪些能力应该被设计成开关 / 权限 / 状态机**
3. **未来从 demo 走向正式自主运行时，哪些 live 控制位应该提前留位**

1. 当前工程事实边界（必须继承）

本计划必须继承当前项目已经被验证通过的工程事实，不得绕开。

当前章节事实

- I = canonical closed，但只是 shadow-only decision-lease control plane closed
- J = functional_closeout_ready_shadow_only
- K = functional_closeout_ready_design_only_gate_closed

当前运行时边界

- system_mode = read_only
- execution_state = disabled

正确解释

当前 GUI / 控制平面设计必须默认服务于： - 受保护运行态 - demo 之前的受控操作态 - demo 期间的可审计运行态 - future live 的预留控制架构

错误解释（禁止）

本计划绝不能被理解为： - 当前就要直接开放 live trading - 当前就允许 Agent 自行修改 live 配置 - 当前就允许 GUI 一键真实放权

2. 总体产品定位

这个系统不应被定义成一个简单前端，而应定义为：

OpenClaw Operator Console + Learning Cockpit

也就是同时覆盖：

- **控制**：查看状态、recheck、arm、enable、relock、kill
- **权限**：Bybit API / 账户权限事实层 + OpenClaw 能力上线层
- **经营**：收益、成本、净收益、趋势、成本归因
- **感知**：延迟、timeout、健康、数据新鲜度、瓶颈、错失机会
- **汇报**：日报、周报、成功/失败复盘、运营总结
- **学习**：经验沉淀、假设生成、实验提案、结果比较
- **审计**：谁做了什么、什么时候做的、产生了什么结果

一句话定义：

这是一个让人类 Operator 和 Agent 一起看见系统、控制系统、理解系统、记录系统、学习系统的运营驾驶舱。

3. 已经确定的设计原则

原则 1：不能继续长期依赖纯 Shell

当前全靠 shell 的方式不可持续。GUI 的存在目的之一，就是把高频、关键、可审计的操作从 shell 迁移到受控的图形交互层。

原则 2：GUI 不是 shell 的外壳

不应把 GUI 设计成“网页版命令行”。正确做法是：- GUI 负责展示和交互 - 后端 Control API 负责校验、执行、审计 - 现有 builders / runners / checks 继续作为底层逻辑资产

原则 3：收益必须看净收益

不能只显示 trading PnL，必须同时显示：- gross PnL - total cost - net PnL

原则 4：总结必须区分事实 / 推断 / 假设

Agent 的汇报、学习、经验沉淀、失败归因，都必须显式区分：- 事实 - 推断 - 假设

原则 5：学习不等于自作主张

Agent 可以：- 观察 - 记忆 - 总结 - 提假设 - 提实验

Agent 不可以：- 自动改 live 配置 - 自动开放 execution - 自动推广未验证策略 - 自动修改代码并直接上线

原则 6：产品族不能被预先局限

Agent 未来不应被固定在单一交易类别。因此 GUI 和开关体系不能只围绕“现货”或“某一种单一产品”设计，而应按**产品族抽象**来设计。

原则 7：按“Bybit 权限边界 × OpenClaw 能力上线边界”设计开关

这已经确定为当前最正确的组织方式。

4. 本计划要覆盖的两条核心主线

主线 A：Bybit API / 账户权限事实层

这条线回答：

交易所 + 当前账户，从权限角度，到底允许做什么？

它反映的是“现实允许边界”，不是 Agent 主观想做什么。

主线 B：OpenClaw 能力上线层

这条线回答：

即便交易所允许，OpenClaw 现在有没有能力安全做这件事？

它反映的是“Agent 当前工程能力边界”。

最终 GUI / Control Plane 必须同时展示两层：- 现实权限层 - Agent 能力层

这样才能区分：- 是交易所不允许 - 还是 Agent 还没做完

5. Capability & Permission 总体开关架构

本计划确定使用以下 7 层开关 / 状态结构。

5.1 Product Family Layer / 产品族层

未来 GUI 与控制平面必须支持按产品族进行能力与权限管理。

第一版建议定义以下产品族： - `spot` - `margin` - `perp_linear` - `perp_inverse` - `options` - `other_derivatives_reserved`

说明：- 这些是“产品族抽象”，不是最终硬编码的 UI 限制。 - 以后即使新增其他类别，也应按产品族扩展，而不是推翻架构。

5.2 Exchange / Account Permission Layer / 交易所与账户权限层

这一层是“事实层”，用于表达 Bybit API 与账户当前具备什么权限。

建议至少包含：

读取权限

- market_data_read_allowed
- account_balance_read_allowed
- positions_read_allowed
- orders_read_allowed
- execution_history_read_allowed
- transfer_history_read_allowed

写入权限

- place_order_allowed
- cancel_order_allowed
- amend_order_allowed
- set_leverage_allowed
- switch_position_mode_allowed
- borrow_repay_allowed
- fund_transfer_allowed

产品族权限

- spot_allowed
- margin_allowed
- perp_linear_allowed
- perp_inverse_allowed
- options_allowed
- other_derivatives_allowed

说明：- GUI 应优先把这一层显示成“系统读取到的当前权限事实” - 这层不是主要靠人工切换，而是靠配置检测 / API 测试 / 权限映射得出

5.3 OpenClaw Capability Layer / OpenClaw 能力上线层

这一层表达：即便交易所允许，Agent 当前到底上线到了哪一步。

每个产品族建议统一使用以下能力等级： - unsupported - observe_only - shadow_ready - demo_ready - live_guarded_ready - live_ready

示例： - spot = demo_ready - perp_linear = shadow_ready - options = unsupported

说明：- 这是最核心的“Agent 实际能力边界” - GUI 必须把这层和交易所权限层并排显示

5.4 Mode Layer / 运行模式层

这一层表达当前系统或某产品族当前处于什么操作模式。

建议统一采用： - disabled - observe_only - shadow_only - demo_enabled - live_blocked - live_guarded - live_enabled

说明： - 既需要支持 **global mode** - 也需要支持 **product-family mode**

这样未来可以出现： - 全局仍然 live_blocked - 但 spot = demo_enabled - perp_linear = shadow_only

5.5 Action Permission Layer / 动作权限层

未来正式自主交易时，不应只有“允许交易/不允许交易”这种单一总开关。

建议按动作拆分： - new_order_allowed - cancel_allowed - amend_allowed - reduce_only_allowed - increase_position_allowed - close_position_allowed - leverage_change_allowed - borrow_repay_action_allowed - fund_transfer_action_allowed

说明： - 未来非常可能需要“允许减仓，不允许加仓”的模式 - 所以动作层必须单独存在

5.6 Risk Envelope Layer / 风险包络层

这一层用于定义每个产品族 / 策略族的风险开关与阈值。

建议纳入： - max_single_trade_risk - max_product_exposure - max_account_exposure - daily_loss_cap - max_consecutive_losses - max_slippage_tolerance - max_latency_tolerance - max_exchange_timeout_tolerance

达到阈值后的动作策略： - alert_only - block_new_orders - reduce_only - relock - global_safe_mode

5.7 Health Gate Layer / 健康门槛层

这一层用于回答：即使策略想做、权限也允许，但系统当前健康到值得做吗？

建议纳入： - ai_latency_gate_enabled - exchange_timeout_gate_enabled - ws_disconnect_gate_enabled - infra_health_gate_enabled - data_freshness_gate_enabled

建议每个门槛同时支持： - threshold - current value - action when exceeded

5.8 Learning & Experiment Layer / 学习与实验层

这是 L 章节前置能力的挂载点。

建议第一版纳入以下开关位： - `learning_enabled` - `observation_feed_enabled` - `lessons_memory_enabled` - `hypothesis_generation_enabled` - `experiment_proposal_enabled` - `replay_experiment_enabled` - `demo_experiment_enabled` - `live_auto_promotion_enabled = false` (默认必须禁用)

说明：- 这层是“允许学习与实验”，不是“允许自动放权”。

6. GUI 现在确定要实现的功能（整合当前全部讨论）

以下内容已经讨论过，并在本计划中正式确认为 GUI 要实现的功能范围。

6.1 Overview / 总览首页

必须显示：- 全局状态 - I / J / K 当前状态 - 今日 gross PnL - 今日 total cost - 今日 net PnL - health score - 当前最危险的 3 个问题 - 当前最值得验证的 3 个假设 - 当前最值得执行的 3 个动作建议

6.2 Control Center / 控制中心

必须包含：- J canonical recheck - K canonical recheck - J functional closeout recheck - K functional closeout recheck - safe recheck group run - demo control state machine - relock / kill 按钮 - operator acknowledgement

说明：- 多重确认（多人批准）当前可以忽略，因为现在就是单人操作单账户 - 时间冷却机制保留设计位，但不强制第一版启用

6.3 Demo Control / Demo Trading 控制

第一版 demo 控制必须支持：- `closed` - `armed_but_closed` - `demo_enabled` - `relocked`

必须支持动作：- validate prerequisites - arm demo - enable demo - relock demo

禁止：- 裸 toggle - 没有前置校验的直接 enable

6.4 Future Live Trading Control / 未来正式运行控制

这部分必须从第一版就开始设计，但全部保持占位或禁用。

必须预留：- live master state - action permissions - autonomy level - product-family scope control - risk envelope settings - health gates - audit / approval display - emergency relock / reduce-only / global stop

说明：- 当前不开放 live - 但未来 live 所需的控制骨架必须提前设计

6.5 Business & Cost / 收益与成本看板

必须支持： - 今日 gross PnL - 今日 total cost - 今日 net PnL - 周 / 月趋势 - 成本拆分 - 成本归因

成本至少包括： - trading fee - funding / borrowing - AI API cost - infra / hardware / hosting - network - manual cost input

6.6 Quick Input / 快速录入

必须支持以下录入：

成本录入

- 日期
- 成本类型
- 金额
- 币种
- 一次性 / recurring
- vendor/source
- notes

事件录入

- 时间
- 事件类型
- 严重程度
- 影响区域
- 备注

配置变更录入

- 生效时间
- 变更项
- 旧值摘要
- 新值摘要
- 操作人
- 是否已验证
- 备注

人工备注录入

例如： - 某天网络异常来自家庭线路问题 - 某次成本增加是一次性硬件采购 - 某次延迟 spike 是 provider 侧问题

6.7 Health & Self-Sensing / 自我感知与健康中心

必须支持： - AI latency - exchange latency - Bybit timeout count - websocket disconnect count - queue backlog - CPU / memory / disk / network - data freshness - stale latest artifact detection - missed opportunity count - estimated missed PnL range - top miss reasons

目标是让 GUI 能回答： - 系统是否健康 - 慢在哪里 - 卡在哪里 - 最近表现变差是不是硬件 / 网络 / API / AI 响应问题导致

6.8 Reports / 报告中心

必须支持：

日报

回答： - 今天赚没赚钱 - 今天总成本是多少 - 今天净收益是多少 - 今天系统健康吗 - 今天有哪些异常 - 明天最需要关注什么

周报

回答： - 本周净收益 - 成本趋势 - 健康趋势 - 成功经验 - 失败模式 - 下周最值得优化的点

成功 / 失败复盘

必须区分： - 成功是因为什么 - 失败是因为什么 - 是市场原因、系统原因、基础设施原因、成本原因、还是配置原因

所有报告必须明确分层： - 事实 - 推断 - 假设

6.9 Learning Center / 自学习中心

这是已确定要纳入 GUI 的功能，不再只是未来可选项。

第一版至少应有： - Observation Feed - Lessons Memory - Hypothesis Queue

后续增强应有： - Experiment Queue - Replay / Demo 实验追踪 - 批准 / 拒绝实验 - 结果对比

说明： - 这是 L 章节前半部分能力的前置落地 - 但不等于允许 Agent 自作主张

6.10 Capability Matrix / 能力矩阵

必须支持： - I / J / K 当前状态 - J decision / contract / closeout - K intake / decision / capability families / contracts / closeout - 每个模块的 latest / contract / blockers / last updated

K 的七条能力族必须显式列出： - adapter - lifecycle - projection - risk - audit - operator switch - acceptance

6.11 Audit Log / 审计日志

必须记录： - 谁做了什么 - 何时做的 - 动作前状态 - 动作后状态 - 成功/失败 - 失败原因 - 关联 artifact / 报告 / runtime latest

必须审计的动作至少包括： - recheck - demo arm / enable / relock - 配置切换 - 成本录入修改 - 事件录入修改 - 学习实验审批 / 拒绝 / 终止

6.12 Settings & Config / 配置与连接管理

GUI 需要支持的，不是“原始 secrets 编辑器”，而是安全的配置操作入口。

第一版建议支持： - provider profile 切换 - API connection test - current active profile display - config rollout state - config version display

第一版不建议支持： - 明文 secrets 管理 - 任意 raw JSON 编辑 - 任意 shell 执行入口

7. 自主 Agent / L 章节前置能力，当前正式纳入哪些

本计划确认将 L 的以下能力前置进入 GUI / Control Plane：

L0: Self-Observation

Agent 能观察自己今天表现如何、哪里慢、哪里错、哪里漏。

L1: Lessons Memory

Agent 能把经验沉淀成结构化知识，而不是每天重新说一遍。

L2: Hypothesis Generation

Agent 能基于事实提出“可能原因 / 可验证改进假设”。

L3: Experiment Proposal

Agent 能把改进想法整理成受控实验提案。

L4: Approval-Dependent Progression

实验推进仍需要人工批准。

当前明确不做： - live self-modification - autonomous live rollout - autonomous code mutation - autonomous execution authority promotion

8. 自主等级（Autonomy Level）规划

未来正式运行时，不应该只有“自动 / 不自动”两档。

本计划建议预留四档： - manual_only - assistive - semi_auto - full_auto

说明： - `manual_only`：只观察 / 给建议，不执行 - `assistive`：给建议，人批准执行 - `semi_auto`：低风险动作自动，高风险动作仍需批准 - `full_auto`：在明确授权边界内完全自主执行

第一版 GUI 可以先显示和占位，不必启用所有档位。

9. 时间冷却（Cooldown）策略

当前结论： - 不作为第一版必须启用项 - 但必须保留设计位

未来可用于： - relock 后短时间内禁止重开 - 从 blocked 切到 guarded 后要求观察窗口 - 风险参数修改后要求稳定窗口 - 新配置切换后要求 shadow/demo 观察

10. 不需要优先做的内容（当前已明确）

以下内容当前不应作为 GUI 第一版重点： - 多重确认 / 多人批准（当前单人单账户场景可忽略） - 明文 secrets 管理 - 任意 shell 面板 - 任意 raw JSON 编辑器 - 完整 live 自动放权 - Agent 自动改代码直接上线

11. 后端架构要求

本计划正式确定：

不建议

- GUI 直接调 shell

建议架构

Layer 1: GUI / OpenClaw front-end

负责： - 页面 - 图表 - 表单 - 交互 - 状态展示

Layer 2: Control API / Backend service

负责： - 读取 latest JSON - 跑受控 recheck - 校验 prerequisites - 触发 arm / relock / safe control actions - 写审计

Layer 3: Data & Analytics layer

负责： - PnL 聚合 - cost 聚合 - telemetry 聚合 - report generation - lessons / hypothesis / experiments storage

12. GUI 第一版 MVP 范围（当前建议）

本计划建议的 MVP 顺序如下：

MVP 第一组（必须最先做）

1. Overview
2. Control Center
3. Capability Matrix
4. Quick Input
5. Audit Log

MVP 第二组（紧接着做）

1. Health & Self-Sensing
2. Business & Cost
3. Reports

MVP 第三组（Learning v1）

1. Learning Center
2. Observation Feed
3. Lessons Memory
4. Hypothesis Queue

MVP 第四组（预留）

1. Future Live Control Architecture（只占位 / 禁用）

13. 最关键的 UI 文案要求

所有关键状态卡片都必须同时说明：- 当前是什么 - 这意味着什么 - 这不意味着什么

例如：- K functional closeout ready 必须同时写：- This does not mean demo trading is enabled. -
operator switch model defined 必须同时写：- Operator enable is still unavailable. -
live control unavailable 必须写：- Reserved for future guarded live architecture.

这样才能防止图形界面本身制造误解。

14. 一句话总括

OpenClaw / Bybit Capability & Permission Switch Plan v1 的核心，不是做几个开关按钮，而是建立一套能同时表达“交易所允许什么、Agent 会做什么、当前模式是什么、哪些动作可做、风险是否允许、系统是否健康、学习是否受控”的统一控制骨架。

这套骨架既服务当前 demo 之前的安全推进，也服务未来正式自主运行时的架构升级。

15. 本文之后的建议下一步

如果采用本计划，建议后续按以下顺序深化：

1. 先整理 **数据字典 / 状态字典**
2. 再整理 **API 清单 / Control API v1**
3. 再整理 **页面级功能草图**
4. 最后进入 GUI MVP 实现

这样后续深化时，就不会忘记当前已经确定下来的功能和要求。